

DAY 19 — PostgreSQL Normalization & Database Design

Today we move from:

Day 16 → Constraints

Day 17 → Indexes

Day 18 → Transactions

Day 19 → Database Design

The main question is:

How should I structure tables so the database remains correct, maintainable, and scalable?

For your GenAI/RAG path, this is especially important because you'll eventually design systems containing:

users

documents

chunks

embeddings

conversations

messages

retrievals

evaluations

ingestion_jobs

Today is about deciding **what belongs in which table**.

1. What Is Database Normalization?

Normalization is a method of organizing relational data to reduce:

- unnecessary duplication
- update anomalies
- insert anomalies
- delete anomalies
- inconsistent data

Consider this table:

documents

document_id	title	user_id	user_name	user_email
-------------	-------	---------	-----------	------------

-----+-----+-----+-----+-----

101	RAG	1	Aarav	a@x.com
-----	-----	---	-------	---------

102	SQL	1	Aarav	a@x.com
-----	-----	---	-------	---------

103 | API | 2 | Emma | e@x.com

Notice:

Aarav

a@x.com

is repeated.

If Aarav changes his email, you may need to update multiple rows.

That's a normalization problem.

2. Better Design

Separate entities:

users

id | name | email

---+-----+-----

1 | Aarav | a@x.com

2 | Emma | e@x.com

and:

documents

id | user_id | title

---+-----+-----

101 | 1 | RAG

102 | 1 | SQL

103 | 2 | API

Relationship:

users

|

| 1

|

└──< documents

Now user information exists in one place.

3. Why Duplication Is Dangerous

Suppose:

documents

id | user_id | user_email

---+-----+-----

101 | 1 | old@example.com

102 | 1 | old@example.com

103 | 1 | old@example.com

Someone updates only row 101:

101 → new@example.com

102 → old@example.com

103 → old@example.com

Now your database contains contradictory information about the same user.

This is called an:

Update anomaly

4. Three Classic Anomalies

You should know:

INSERT anomaly

UPDATE anomaly

DELETE anomaly

These are major reasons normalization exists.

5. Update Anomaly

Suppose:

documents

document_id | user_id | user_name

-----+-----+-----

101 | 1 | Aarav

102 | 1 | Aarav

103 | 1 | Aarav

Aarav changes his name.

If you update:

101

102

but forget:

103

you now have:

Aarav

Aarav Kumar
for the same user.

6. Insert Anomaly

Imagine:

document_id

user_id

user_name

document_title

Suppose a new user registers but hasn't uploaded a document.

How do you insert the user?

You may be forced to insert:

NULL document

or duplicate unrelated information.

That's an **insert anomaly**.

With separate tables:

users

can store the new user independently.

7. Delete Anomaly

Suppose:

documents

document_id | user_id | user_name

-----+-----+-----

101 | 1 | Aarav

If you delete document 101 and this was the only row containing Aarav's information, you accidentally lose information about the user.

That's a:

Delete anomaly

Separate users and documents tables prevent this.

8. First Normal Form — 1NF

At a practical level, 1NF means values should be atomic rather than storing multiple independent values in one cell.

Bad:

users

id | name | phone_numbers

---+-----+-----

1 | Aarav | 9876, 8765, 7654

The column contains multiple values.

Better:

users

id | name

---+-----

1 | Aarav

and:

user_phones

user_id | phone

-----+-----

1 | 9876

1 | 8765

1 | 7654

Relationship:

users

|

└───< user_phones

9. Another 1NF Example

Bad:

document

id | chunks

---+-----

101 | chunk1, chunk2, chunk3

Better:

documents

id

101

and:

document_chunks

document_id | chunk_number | content

-----+-----+-----

101 | 1 | ...

101 | 2 | ...

101 | 3 | ...

This is exactly how your RAG database should normally model chunks.

10. What Does "Atomic" Mean?

Atomic doesn't necessarily mean:

"The value can never contain spaces or punctuation."

It means the column represents a single logical value for the relational design.

For example:

full_name = 'Aarav Kumar'

can be perfectly reasonable as one attribute.

But:

phone_numbers = '123,456,789'

may represent a repeating group and should often be modeled separately.

11. Second Normal Form — 2NF

2NF matters particularly when you have a **composite primary key**.

A table is in 2NF when:

It is in 1NF and non-key attributes depend on the whole primary key rather than only part of it.

Consider:

document_chunks

document_id

chunk_number

document_title

content

with:

PRIMARY KEY (

document_id,

chunk_number

)

Now:

content

depends on:

(document_id, chunk_number)

But:

document_title

depends only on:

document_id

not on:

chunk_number

That's a partial dependency.

12. Fixing the 2NF Problem

Separate:

documents

document_id | title

-----+-----

101 | RAG Guide

and:

document_chunks

document_id | chunk_number | content

-----+-----+-----

101 | 1 | ...

101 | 2 | ...

Now:

document title

belongs to the document.

chunk content

belongs to the chunk.

Much cleaner.

13. Third Normal Form — 3NF

3NF deals with **transitive dependencies**.

A simplified mental model:

Non-key attributes should depend on the key, the whole key, and nothing but the key.

Consider:

users

user_id

name

country

country_name

If:

country → country_name

then:

user_id → country → country_name

country_name is indirectly dependent on user_id.

That may be a normalization problem.

14. Practical 3NF Design

Instead of:

users

id | name | country_code | country_name

you could have:

users

id | name | country_code

and:

countries

country_code | country_name

-----+-----

IN | India

US | United States

UK | United Kingdom

Then:

users.country_code

↓

countries.country_code

15. Normalization Mental Model

Think:

1NF

↓

Atomic values / no repeating groups

2NF

↓

No partial dependency on composite key

3NF

↓

No unnecessary transitive dependency

For most application database design, understanding these concepts is more useful than memorizing textbook definitions.

16. Normalized RAG Schema

A reasonably normalized RAG system might look like:

users

|

|—————< documents

| |

| L————< document_chunks

| |

| L————< embeddings

|

|—————< conversations

|

|————< messages

Let's break it down.

17. Users

```
CREATE TABLE users (  
  id BIGINT PRIMARY KEY,  
  name TEXT NOT NULL,
```

```
email TEXT NOT NULL UNIQUE,  
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

One row = one user.

18. Documents

```
CREATE TABLE documents (  
id BIGINT PRIMARY KEY,  
user_id BIGINT NOT NULL,  
title TEXT NOT NULL,  
status TEXT NOT NULL,  
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
FOREIGN KEY (user_id)  
REFERENCES users(id)  
);
```

One row = one document.

19. Document Chunks

```
CREATE TABLE document_chunks (  
document_id BIGINT NOT NULL,  
chunk_number INTEGER NOT NULL,  
content TEXT NOT NULL,  
  
PRIMARY KEY (  
document_id,  
chunk_number  
),
```

```
FOREIGN KEY (document_id)  
REFERENCES documents(id)  
);
```

One row = one chunk.

20. Embeddings

Potential design:

```
CREATE TABLE embeddings (  

```

```
id BIGINT PRIMARY KEY,  
chunk_id BIGINT NOT NULL,  
model TEXT NOT NULL,  
vector_data TEXT,  
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```
FOREIGN KEY (chunk_id)  
REFERENCES document_chunks(...);  
);
```

In an actual vector-enabled PostgreSQL system, you'd typically use a vector-capable extension/type rather than TEXT for the vector itself.

The important design point is:

chunk

↓

embedding

rather than storing an arbitrary number of embeddings inside one chunk row.

21. One-to-One vs One-to-Many

Database design requires understanding cardinality.

One-to-one

user

↓

profile

Potentially:

users.id

↓

profiles.user_id UNIQUE

The UNIQUE constraint prevents multiple profiles for one user.

One-to-many

user

↓

documents

One user can have many documents.

No UNIQUE on:

documents.user_id

Many-to-many

Example:

documents



tags

A document can have many tags.

A tag can belong to many documents.

You normally need a junction table:

document_tags

document_id | tag_id

-----+-----

101 | 1

101 | 2

102 | 1

22. Many-to-Many Design

Tables:

documents

tags

document_tags

Example:

```
CREATE TABLE tags (  
id BIGINT PRIMARY KEY,  
name TEXT NOT NULL UNIQUE  
);
```

Junction table:

```
CREATE TABLE document_tags (  
document_id BIGINT NOT NULL,  
tag_id BIGINT NOT NULL,
```

```
PRIMARY KEY (  
document_id,  
tag_id  
),
```

```
FOREIGN KEY (document_id)
```

REFERENCES documents(id),

FOREIGN KEY (tag_id)

REFERENCES tags(id)

);

Notice:

(document_id, tag_id)

is a natural composite primary key.

23. Why Not Store Tags as a String?

Bad:

documents

id | tags

---+-----

101 | rag,llm,postgres

Problems:

hard to query

hard to enforce uniqueness

hard to join

hard to maintain

A normalized relational model gives:

documents

↓

document_tags

↓

tags

24. Normalization vs Performance

This is where real engineering begins.

Highly normalized data may require many joins.

For example:

users

↓

documents

↓

chunks

↓

embeddings

A query may require multiple joins.

You might eventually choose some denormalization for performance.

For example:

document_metrics

document_id

chunk_count

embedding_count

successful_embeddings

But now you have duplicated/derived information.

That introduces consistency questions.

25. Denormalization

Denormalization means intentionally storing redundant/derived data to improve:

- read performance
- reporting
- simplicity of expensive queries
- analytical workloads

Example:

Normalized:

documents

document_chunks

embeddings

To calculate:

chunk_count

you might run:

```
SELECT COUNT(*)  
FROM document_chunks  
WHERE document_id = 101;
```

Denormalized:

documents

id | title | chunk_count

---+-----+-----

Now reads are faster.

But:

Who updates chunk_count?

That's the tradeoff.

26. Normalization vs Denormalization

Think:

Normalization

↓

less duplication

better integrity

more joins

Denormalization

↓

more duplication

potentially faster reads

more consistency complexity

Neither is automatically "better."

It depends on workload.

27. OLTP vs Analytics

This is very important for Data Engineering.

OLTP

Online transactional workloads:

users

documents

orders

payments

Usually favor:

normalized schema

because correctness and transactional updates matter.

OLAP / analytics

Analytical workloads often benefit from:

denormalized structures

fact tables

dimension tables

materialized views

aggregations

You'll encounter this more deeply in Data Engineering.

28. RAG Production Example

Imagine your API frequently needs:

user

document count

chunk count

embedding count

last ingestion status

Computing this from raw tables every request could be expensive.

You might build:

user_rag_metrics

containing:

user_id

document_count

chunk_count

embedding_count

failed_documents

last_ingestion_at

This is a **derived/denormalized structure**.

But your source of truth remains:

users

documents

chunks

embeddings

This distinction is important.

29. Source of Truth

Suppose:

documents.page_count = 50

and:

user_metrics.total_pages = 50

The second value is derived.

If it says:

`user_metrics.total_pages = 70`

which one is authoritative?

Usually:

`documents`

is the source of truth.

The metric table must be refreshed/recomputed correctly.

30. Surrogate vs Natural Keys

Another important design concept.

Natural key

A real-world attribute:

email

ISBN

country_code

Surrogate key

Artificial identifier:

BIGINT ID

UUID

Example:

`id BIGINT PRIMARY KEY`

The ID doesn't represent business meaning.

31. Why Surrogate Keys Are Common

Suppose email changes:

old:

`aarav@example.com`

new:

`aarav.dev@example.com`

If email were the primary key, many relationships become more complicated.

Using:

`user_id = 1001`

provides a stable identifier.

This is common in application systems.

32. Natural Keys Can Still Be Useful

You might still enforce:

email TEXT UNIQUE

while using:

id BIGINT PRIMARY KEY

So:

PRIMARY KEY → technical identity

UNIQUE → business uniqueness

This is a common design.

33. Soft Delete

Production systems sometimes avoid physically deleting rows.

Instead:

deleted_at TIMESTAMPTZ

Then:

deleted_at IS NULL

means active.

Example:

```
SELECT *  
FROM documents  
WHERE deleted_at IS NULL;
```

This allows historical recovery/auditing.

But soft deletes introduce complexity:

every query may need deleted_at filtering

unique constraints become more complicated

storage keeps growing

relationships remain

So don't use soft delete automatically.

34. Temporal / Audit Design

Production systems often need:

created_at

updated_at

deleted_at

Example:

created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()

Be careful:

DEFAULT NOW() sets the initial value.

It does **not** automatically update updated_at every time the row changes.

You need application logic or a database mechanism such as a trigger to maintain it.

DAY 19 DIFFICULT PRACTICE

Now you design.

Challenge 1 — Identify the Anomaly

Consider:

documents

document_id

document_title

user_id

user_name

user_email

country

country_name

Example:

101 | RAG Guide | 1 | Aarav | a@x.com | IN | India

102 | SQL Guide | 1 | Aarav | a@x.com | IN | India

103 | API Guide | 2 | Emma | e@x.com | US | USA

Identify:

1. At least two duplicated pieces of information.
 2. An update anomaly.
 3. An insert anomaly.
 4. A delete anomaly.
 5. How you would redesign it.
-

Challenge 2 — Normalize a RAG Table

You inherit:

document_data

document_id

document_title

user_id
user_name
user_email
chunk1
chunk2
chunk3
embedding1
embedding2
embedding3

This is obviously problematic.

Design a normalized schema.

At minimum, identify the tables you would create and their relationships.

Don't worry about writing every column yet.

Challenge 3 — 1NF

Is this table in a good 1NF-style relational design?

documents

id | title | tags

---+-----+-----

101 | RAG | rag,llm,postgres

If not, redesign it.

Challenge 4 — 2NF

Consider:

document_chunks

document_id
chunk_number
document_title
chunk_content

PRIMARY KEY (
document_id,
chunk_number
)

Which attribute violates the idea behind 2NF?

Explain why.

Then redesign the tables.

Challenge 5 — 3NF

Consider:

users

user_id

user_name

country_code

country_name

Assume:

country_code → country_name

Explain the dependency chain.

Then propose a 3NF-style redesign.

Challenge 6 — Cardinality

For each relationship, identify whether it is:

1:1

1:N

N:M

A

One user can upload many documents.

B

A document has many chunks.

C

A document can have many tags and a tag can belong to many documents.

D

One user has exactly one profile.

Challenge 7 — Many-to-Many

Design:

documents

tags

document_tags

Requirements:

- document ID
- tag ID
- tag name

- no duplicate document/tag relationship
- foreign keys
- appropriate primary key

Write the SQL.

Challenge 8 — Normalize vs Denormalize

You have:

documents = 500 million

document_chunks = 5 billion

embeddings = 5 billion

Your API frequently asks:

"Give me document title + chunk count + embedding count."

Would you:

- A. Always normalize
- B. Always denormalize
- C. Consider a derived/denormalized structure

Choose one and explain the tradeoff.

Don't treat this as a memorization question.

Challenge 9 — Production Schema Design

Design a normalized PostgreSQL schema for:

A RAG application where users upload documents, documents are split into chunks, chunks have embeddings, users have conversations, and conversations contain messages.

You should have at least:

users

documents

document_chunks

embeddings

conversations

messages

For each table identify:

- primary key
- foreign keys
- important NOT NULL
- important UNIQUE
- important CHECK
- cardinality

You don't need to write complete SQL yet.

Focus on the architecture.

Challenge 10 — LEFT JOIN Reasoning

Suppose:

users

documents

are properly normalized.

A developer says:

"Since users and documents are normalized, we should always use INNER JOIN because every document belongs to a user."

Is this reasoning correct?

Explain.

Hint:

Think about:

documents → users

versus:

users → documents

and your Day 15 weakness.

Challenge 11 — Source of Truth

You have:

documents

and:

document_metrics

where:

document_metrics.chunk_count

is derived from:

document_chunks

If these disagree:

document_chunks → 120 rows

document_metrics.chunk_count → 117

Which should generally be treated as the source of truth?

Why?

Challenge 12 — Denormalization Failure

A team stores:

documents

id

title

chunk_count

embedding_count

but does not maintain these values transactionally or through a reliable pipeline.

Six months later:

actual chunks = 100

chunk_count = 87

Explain why this happened.

Give **two ways** to prevent this class of problem.

DEBUGGING

Challenge 13

A developer creates:

users

id

name

email

documents

where documents contains:

101,102,103

inside a single column.

They say:

"This is simpler than another table."

Give at least three problems with this design.

Challenge 14

Someone proposes:

document_chunks

chunk_id PRIMARY KEY

document_id

document_title

chunk_number

content

Explain why storing document_title here may create redundancy.

Would you always reject this design?

Explain your reasoning.

Challenge 15 — Production Tradeoff

You have a normalized schema:

users

documents

chunks

embeddings

The retrieval API performs four joins for every request.

The team proposes:

rag_document_summary

with precomputed:

document_id

user_id

chunk_count

embedding_count

status

Should you automatically reject this because it violates normalization?

Explain the engineering tradeoff.

INTERVIEW ROUND

Answer without searching.

Q16

What is database normalization?

Q17

Why does normalization reduce update anomalies?

Q18

Explain:

INSERT anomaly

UPDATE anomaly

DELETE anomaly

Q19

What is 1NF?

Q20

What problem does 2NF address?

Q21

What problem does 3NF address?

Q22

What is a one-to-many relationship?

Q23

What is a many-to-many relationship?

Q24

Why does a many-to-many relationship usually require a junction table?

Q25

What is denormalization?

Q26

Why might a production system intentionally denormalize data?

Q27

What is the difference between a natural key and a surrogate key?

Q28

Why are surrogate IDs commonly used in application databases?

Q29

What is the difference between a source-of-truth table and a derived/denormalized table?

Q30

When would a materialized/derived metric be useful in a RAG system?

INTERVIEW SCENARIO

Imagine you're interviewing for:

Data Engineer / GenAI Engineer

The interviewer asks:

"Design the PostgreSQL schema for a production RAG platform."

The system needs:

Users

Documents

Chunks

Embeddings

Conversations

Messages

Document tags

Ingestion jobs

The system must support:

1. Document uploads
2. Chunking
3. Embedding generation
4. Retrieval
5. Chat conversations
6. Multiple messages per conversation
7. Failed ingestion retries
8. Document deletion
9. Analytics

Explain your schema design.

I want you to discuss:

Primary keys

Foreign keys

Cardinality

Normalization

Composite keys

Constraints

Indexes

Transactions

ON DELETE behavior

Source of truth

Derived metrics

Don't just list tables.

Explain **why the relationships are designed that way.**

Day 19 Mastery Target

You should now be able to look at a table and ask:

What entity does this row represent?

↓

What is its primary key?

↓

What does each attribute depend on?

↓

Is information unnecessarily duplicated?

↓

What are the relationships?

↓

1:1 / 1:N / N:M?

↓

Where should foreign keys live?

↓

What should be normalized?

↓

What might intentionally be denormalized?

↓

What is the source of truth?

The most important mindset shift:

Normalization isn't "split every table as much as possible."

It's about creating a model where each fact has an appropriate home, while making deliberate tradeoffs when performance or operational requirements justify derived/denormalized structures.

Your assignment

Solve **Challenges 1–15**, then answer **Interview Q16–30** and the production RAG schema scenario.

I'll evaluate your design decisions at your current level rather than restarting from basic SQL.